

Hong Kong Metropolitan University

COMP S209W/8090SEF

Data Structures, Algorithms and Problem Solving
Data Structures, Algorithms

Course Study Report (Task 2)
Group 10

Submission date: __17/2/2026__

I declare that:

- (i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;
- (ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
- (iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
- (iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
- (v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

Name	SID
Ng Chun Yiu	12893986

2. Each student must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

Heap Data Structure and Heap Sort Algorithm

GitHub repository link: <https://github.com/KaBooMU/COMPS209W-12893986/tree/main/COMP-S209W-CourseProject/>

Introduction video link: <https://youtu.be/IDe4b9Th2jQ>

Introduction of the Selected Topics and Their Implementations

This study report presents one new data structure and one new algorithm that are not covered in the core content of the course, namely the heap data structure and the heap sort algorithm. The heap was selected because it provides an efficient way to maintain partially ordered data, while heap sort was selected because it is a classical comparison-based sorting algorithm with a guaranteed time complexity of $O(n \log n)$. The two topics are closely related, which makes them suitable for integrated study: the algorithm is built directly on the data structure, and the code clearly shows how the design of the heap supports the sorting process. The implementation submitted for this task consists of a Python file that defines a MaxHeap class together with standalone functions for heap construction, heap repair and heap sort. This allows the report to discuss both abstract design and concrete implementation in a coherent way.

Heap as an Abstract Data Type (ADT)

A heap is a complete binary tree meeting the property of a heap. In a max heap the parent node is greater than or equal to its children, and in a min heap the parent node is less than or equal to its children. For the implementation used in this project a max heap, as it is a convenient way to compute ascending order from extracting the largest element repeatedly, since it satisfies the properties. The completeness property means that the tree is filled level by level from left to right, so that instead of linking nodes with the full sequence, we could represent it compactly as a Python list. If a node is stored with id i , its left child points to $2i + 1$, its right child will point to $2i + 2$ and its parent is located at $(i - 1) / 2$. The way we display indices prevents us from using any pointers and thus the structure can easily be computed using index arithmetic. The major methods included in our program are `insert`, `extract_max`, `_heapify_up` and `_heapify_down` for the heap class and the helper `heapify` as used by heap sort separately. In combination these methods define the fundamental behavior of the data structure: insertion maintains the order from bottom to top, extraction restores the order from top to bottom and the heap state stays intact even after each modification.

Development of the Data Structure and the Algorithm

The initial phase of the data construction starts with the MaxHeap class that contains elements in the list attribute `heap`. When a value is added, it is added to the end of the list and compared with its parent. If the new value is larger than the old value, the two values are exchanged and the same is done until the max-heap property is reclaimed. For this behaviour the private method `_heapify_up` is present. In such, when the maximum value is found, the root is removed, the last element is moved to the root position, and the root is moved downward until it is larger than or equal to its children. The private method `_heapify_down` takes care of this behavior. The two methods illustrate the evolution of the heap as a functional, rather than a abstract, data structure. It is based on the same principle that the algorithm is developed. To approach this, `build_max_heap` is used to transform a badly arranged array to a proper max heap by applying

heapify from the last internal node back to the root. Secondly, `heap_sort` repeatedly swaps the root with the last unsorted element, thereby reducing the effective heap size and restores the heap property on the reduced prefix. Thus, the data structure and algorithm are directly associated: the sorting routine succeeds since the heap continually reveals the largest remaining value at its root.

How the Algorithm Works

Heap sort is carried out in two stages. In the first phase, we make a max heap from the initial list. The algorithm starts from the last non-leaf node and proceeds up the hierarchy, because the leaf nodes fulfill the heap property. After this stage, the maximum value in the entire list is at index 0. The second portion consistently shifts that maximum value toward its last sorted location at the end of the array. It is deemed that each swap reduces the heap size by one, since the item that was swapped is already stored in the proper position and is no longer in the unsorted region. It then calls heapify on the root to fix the reduced heap. Here the process continues until the whole array is sorted in ascending order. We make a simple comparison in the Python file submitted as a result of converting the original list [34, 12, 56, 78, 23, 9, 45, 1] into [1, 9, 12, 23, 34, 45, 56, 78]. The flowchart and the heap diagram demonstrate a sequence of operations as shown in the appendix.

Time Complexity Analysis

The time complexity of heap sort is $O(n \log n)$ for best case, average case and worst case. The reason is that, in the process of building the max heap, it takes $O(n)$ time and in their sorting phase $n - 1$ is extracted from the heap, with heapify operation with height-limited (the height of the heap is $O(\log n)$) operation is then conducted. Hence running time is based on recursively performing logarithmic repairs over n elements in total. This provides heap sort with an expected upper bound, which is much more preferable to quadratic algorithms such as bubble sort or selection sort for large inputs. As far as space, heap sort is an in-place algorithm as it sorts the list inside the array, but does not need another n list. In the implementation specified here, just heapify recursive calls add a little added stack overhead. A summary of complexity is presented in Table A1.

Suitable Application Examples

The heap data structure can be implemented in many scenarios. The most common was a priority queue in which you need to extract the highest priority element quickly. Heap is also a popular method for scheduling systems, for event-driven simulation and for graph algorithms like Dijkstra's shortest-path method and A* search. Heap sort itself is appropriate when a predictable $O(n \log n)$ worst-case bound is a reasonable approximation and stability is unnecessary. Python already has a `heapq` module for min-heaps in the standard library, but creating the data structure is helpful because it shows knowledge of list representation, tree indexing, modular functions and algorithms. This project provides in so doing, both conceptual insight and program practice.

Appendix A. Figures and Tables for Task 2

Figure A1. Example of a max heap

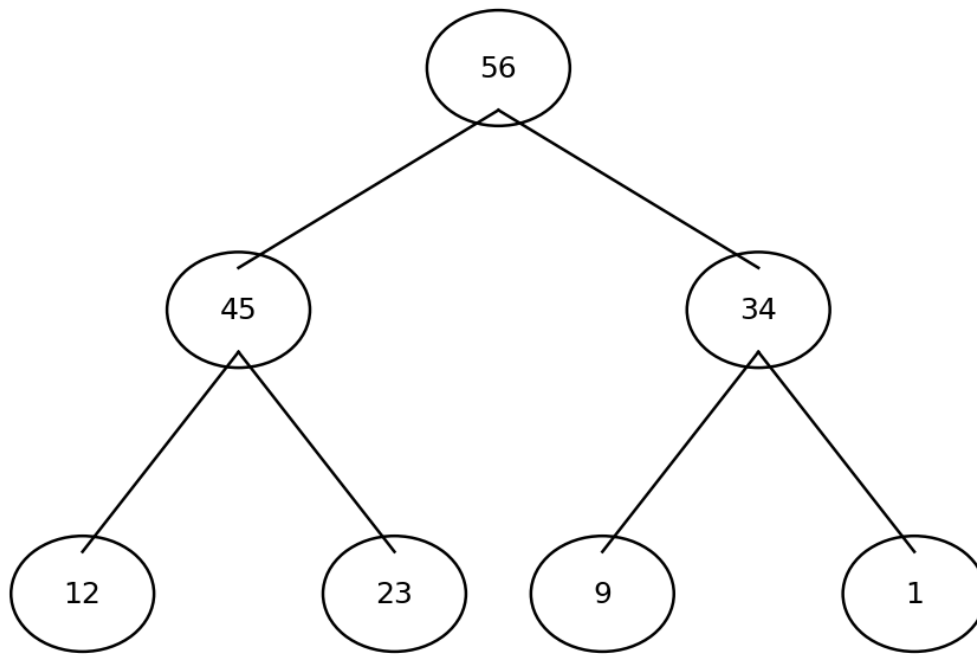


Table A1. Heap sort complexity summary

Case	Time Complexity	Reason
Best case	$O(n \log n)$	Heap construction is linear, but the repeated root swaps still require logarithmic repair.
Average case	$O(n \log n)$	Typical execution follows the same build-and-repair pattern across shrinking heap sizes.
Worst case	$O(n \log n)$	Each extraction is bounded by the height of the heap, even under maximum restructuring.

References

- Python Software Foundation. (n.d.). Python documentation. <https://docs.python.org/3/>
- Williams, J. W. J. (1964). Algorithm 232: Heapsort. Communications of the ACM, 7(6), 347–348.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Hong Kong Metropolitan University. (2026). COMP S209W Data Structures, Algorithms and Problem Solving lecture notes.